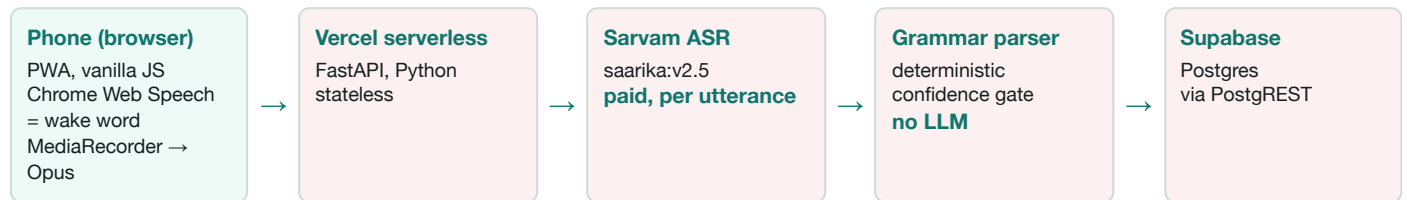


Technical architecture — today, and once funded

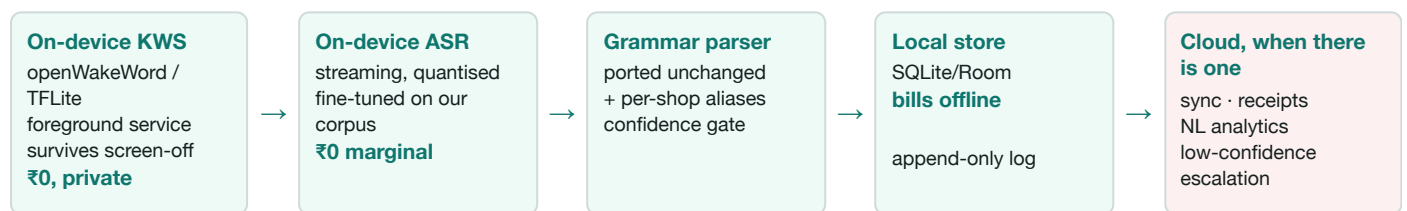
Two states. The difference between them is not features; it is where the computation happens, and that single move is what turns the cost structure from impossible to viable.

A · As validated in the PoC



Every arrow crosses the network. Measured ASR round trip ~2,200 ms against a 1,150 ms p95 target; no network means no billing; and each utterance is a metered call against a ₹10/shop/month ceiling that assumes ₹0 cloud inference. The PoC proved the *grammar* and the *product*. It cannot ship as the business.

B · Target, once funded — Android, device-first



The whole billing path runs on the handset. Latency becomes local, the network becomes optional, and marginal inference cost goes to zero — which is the only way a ₹199/month tier survives contact with 24,000 utterances. Nothing about the grammar, the language packs or the confidence gate changes; they are already pure functions over text.

Where AI earns its place — and where it does not

Technique	Verdict	Why
On-device streaming ASR fine-tuned, quantised	Core	This is the business model, not an optimisation. It removes the metered call, meets the latency target and makes offline possible in one move. The training set is the corrected-utterance corpus — which is why the corpus is listed as a key resource rather than a nice-to-have. VALIDATE target WER on code-mixed Tamil retail speech before committing a hire to it.
On-device keyword spotting	Core	Replaces Chrome's cloud recogniser, which is free but stops when the tab is hidden and goes through Google's servers. A foreground service keeps listening with the screen off — which is what "hands stay on the goods" actually requires.
Deterministic grammar for the billing path	Core (and contrarian)	Deliberately not an LLM. A grammar's failures are legible: when it cannot parse, it knows, and that feeds the confidence gate. An LLM returns something plausible every time, which is precisely the silent-error mode this product cannot survive — a wrong bill destroys trust permanently. This choice will be questioned by investors who expect "AI-native" to mean "LLM everywhere". The answer is that we already found three silent mispricing bugs in testing; a model that always answers confidently would have hidden all three.
LLM function-calling structured intent extraction	Escalation only	Earns its place on the utterances the grammar <i>rejects</i> — not the ones it handles. Roughly the tail, not the trunk. Gated by confidence, capped per shop per day, and never on the happy path, because the cost model cannot carry a cloud call per transaction. VALIDATE what share of real utterances actually fall to the tail — measurable from the utterance log today.
Alias catalog (lightweight RAG)	Core, shipped	Already built and already the moat: the shop teaches the system its own words. Full embedding retrieval earns its place only when catalogs pass a few hundred SKUs — a Tier 3 problem. Using vectors on a 24-SKU café catalog would be AI theatre.
Predictive replenishment	Roadmap, not LLM	Retail demand at kirana scale is intermittent and lumpy — Croston or a simple ETS beats anything generative, runs on the phone, and is explainable to a shopkeeper who has to act on it. The differentiator versus GoFrugal is that they alert on a threshold and we forecast; it is not that we use a bigger model.
Natural-language analytics	Core — promoted	The clearest legitimate LLM use in the product. Text-to-SQL over a deliberately small, fixed schema, answering in the shop's own language. It is cheap (a handful of queries a week, not 24,000), it is bounded (the schema is ours), and it attacks the founder's real point: the owner currently outsources understanding of their own business to an auditor. Failure here is also cheap — a wrong answer to "how much coffee did I sell" is embarrassing, not a wrong bill.
Anomaly detection	Later, statistical first	Void/refund abuse and cash mismatches are real, but at one counter and fifty bills a day the base rates are too low for anything learned. Simple rules and z-scores first; models when there is a chain to compare across.

Hardware abstraction — why the pricing strategy is technically possible

One interface between the app and the physical world, with implementations behind it: microphone (phone mic / headset / terminal mic), printer (Bluetooth Classic SPP / BLE / network 9100 / Android print), scanner (camera / USB HID), display (phone / dual-screen terminal). The billing stack above never learns which it is talking to.

That is what lets the same product run on a ₹10,000 phone the shop already owns and on a ₹90,000 terminal a chain insists on — **the commercial tiering in the pricing model depends on this being true in code, not just on a slide.** It is also the reason the PoC's browser limitation is a browser limitation and not an architectural one: cheap ESC/POS printers speak Bluetooth Classic, which no web page can open, and the abstraction is where that gets solved rather than worked around.

Migration risk, stated plainly

The port is not a rewrite of the hard part. The grammar, the six language packs, the confidence thresholds and 1,561 tests are pure text-processing and move to any runtime intact — that is the single biggest asset the PoC produced. What is genuinely new is the audio pipeline, the local store and sync, and the on-device model. The honest risk is the last of those: **if a small quantised model cannot hit acceptable accuracy on code-mixed retail speech, the ₹0-inference plan fails and pricing has to move.** That is the sensitivity to show investors first, and it is measurable early with the corpus we are already collecting.

Bolo Bill · Technical architecture · current state verified against the running system · **VALIDATE** items are measurable from the existing utterance log · August 2026